

RAZVOJ TERMINAL EDITORA SA POSEBNIM OSOBINAMA

DEVELOPMENT OF A TERMINAL EDITOR WITH SPECIAL FEATURES

Kandidat

ŽELJKO RAŠOVIĆ
26122026

Mentor

DR. PLETL SILVESTER

Subotica, 2026

Izjava studenta

Izjavljujem da završni rad predstavlja rezultat mog sopstvenog rada. Literatura i sredstva koja su korišćena pri izradi rada u potpunosti su citirana. Visoka tehnička škola strukovnih studija u Subotici može za svoje potrebe, bez naknade, da iskoristi rezultate predstavljene u završnom radu, imajući u vidu eventualna ograničenja definisana posebnim sporazumom o tajnosti, koji u tom slučaju čini sastavni deo završnog rada.

U Subotici, _____ 2026. _____ godine

Željko Rašović

Potpis studenta

Sažetak

U završnom radu prikazan je razvoj terminalnog editora otvorenog koda pod nazivom rwx (read, write, execute). Editor je implementiran u programskom jeziku C, a kao polazna osnova korišćen je vodič Build Your Own Text Editor [1].

Naročita pažnja je posvećena primeni suckless filozofije razvoja softvera [2], gde se editor konfiguriše menjanjem izvornog koda pomoću zakrpa (engl. patch), umesto kroz eksterne konfiguracione fajlove ili dinamičke plugin sisteme.

Implementirane su sledeće funkcionalnosti editora: rad u sirovom (engl. raw) režimu terminala, učitavanje i čuvanje fajlova, kretanje kursorom, uređivanje teksta, prikaz statusne linije, pretraga i zamena, undo/redo, file browser, isticanje sintakse određenih programskih jezika, modalno uređivanje, podrška za miš, bracketed paste, detekcija spoljašnje izmene fajla i kopiranje putem OSC52 escape sekvence.

Editor radi na Windows operativnim sistemima i na POSIX sistemima, kao što su Linux, macOS, FreeBSD, OpenBSD i NetBSD.

Uz editor dolaze i pomoćne skripte (configure, compile, install, uninstall, patch_code) koje olakšavaju korisniku konfigurisanje, kompajliranje, instalaciju, uklanjanje i proširenje funkcionalnosti editora. Izrađene su dve referentne zakrpe, zakrpa za isticanje C++ sintakse i zakrpa za vim prečice. U dokumentaciji je dat i primer kako korisnik samostalno može da generiše i primeni sopstvenu zakrpu.

Ključne reči: C programski jezik, suckless filozofija, terminal editor, vim, zakrpa (engl. patch).

Sadržaj

Izjava studenta	2
Sažetak	3
Sadržaj	4
Spisak oznaka	6
Zahvale	7
Zadatak završnog rada	8
1. Uvod	9
1.1. Opis problema	9
1.2. Ciljevi rada	10
2. Teorijske osnove	11
2.1. Analiza i komparativni prikaz postojećih terminalskih editora	11
2.2. Suckless filozofija razvoja softvera	13
2.3. Konfigurisanje kroz izvorni kod naspram konfiguracionih fajlova	14
2.4. Mehanizam zakrpa (engl. patches) kao alternativa plugin ekosistemima	14
3. Rešavanje problema	15
3.1. Arhitektura i organizacija koda	15
3.1.1. Opšta objašnjenja	15
3.2. Terminalni sloj i sirovi režim rada	17
3.3. Struktura podataka i upravljanje memorijom	18
3.4. Učitavanje, čuvanje i detekcija eksterne izmene fajla	20
3.5. Modalno uređivanje i obrada korisničkog ulaza	21
3.6. Pretraga, zamena i navigacija	23
3.7. Isticanje sintakse i renderovanje teksta	29
3.8. Pretraživač fajlova	31
3.9. Undo/redo i izbor teksta	33
3.10. Merenje performansi	34
3.11. Dokumentacija i objavljivanje koda	35
3.12. Pomoćne skripte za razvoj, instalaciju i održavanje	35
3.12.1. Skripta configure	36
3.12.2. Skripta compile	36

3.12.3. Skripta install	37
3.12.4. Skripta uninstall	38
3.12.5. Skripta patch_code	38
3.12.6. Skripta add_path_windows.bat	38
3.12.7. Primer kompletnog rada sa skriptama	39
3.13. Zakrpe za proširenje funkcionalnosti	40
3.13.1. Zakrpa za C++ sintaksu	40
3.13.2. Testni fajlovi za sintaksu	41
3.13.3. Zakrpa za vim prečice	41
4. Zaključak	42
Literatura	43
5. Prilozi	44
5.1. Štampani materijali	44
5.2. Elektronski materijali	44

Spisak oznaka

Oznaka	Značenje
ANSI	American National Standards Institute, standard za terminalne escape sekvence
API	Programski interfejs aplikacije (engl. Application Programming Interface)
C	Programski jezik
IDE	Integrisano razvojno okruženje (engl. Integrated Development Environment)
LOC	Broj linija koda (engl. Lines of Code)
OSC	Operating System Command, standard za terminalne escape sekvence za komunikaciju sa terminalom
POSIX	Portable Operating System Interface
RAM	Radna memorija (engl. Random Access Memory)
SGR	Select Graphic Rendition, ANSI komande za boje i stilove teksta
UTF-8	Univerzalni kodirajući format

Zahvale

Zahvaljujem se svim profesorima i zaposlenima Visoke tehničke škole strukovnih studija u Subotici, koji su svojim znanjem, iskustvom i posvećenošću doprineli sticanju stručnih znanja i veština koje predstavljaju osnovu mog daljeg profesionalnog razvoja.

Zadatak završnog rada

U okviru završnog rada potrebno je:

1. Izvršiti analizu postojećih terminalskih editora teksta (vi, vim, neovim, nano, kilo) i uporediti njihove arhitekture, način konfigurisanja i proširivanja.
2. Opisati suckless filozofiju razvoja softvera i mehanizam patch-ovanja izvornog koda kao alternativu klasičnim plugin-ima i konfiguracionim fajlovima.
3. Projektovati arhitekturu editora rwx sa jasnim odvajanjem jezgra (rad sa terminalom, baferom teksta, ulaznim događajima) od proširivih komponenti.
4. Implementirati osnovne funkcionalnosti editora u programskom jeziku C: rad u sirovom (raw) režimu terminala, učitavanje i čuvanje fajla, kretanje kursorom, uređivanje teksta, pretragu i prikaz statusne linije.
5. Razviti dodatne funkcionalnosti prema potrebama korisnika (npr. isticanje sintakse, modalno uređivanje, prečice po uzoru na vim i sl.) i realizovati ih u obliku zasebnih patch-eva.
6. Izraditi referentni patch koji služi kao primer i uputstvo budućim korisnicima kako da kreiraju sopstvene patch-eve (plugin-ove) za editor.
7. Testirati editor na nekoliko realnih scenarija upotrebe i izvršiti merenje veličine binarnog izlaza i potrošnje resursa.
8. Pripremiti dokumentaciju projekta (README, uputstvo za prevođenje i primenu patch-eva) i objaviti izvorni kod na javnom Git repozitorijumu.
9. Izvesti zaključke o prednostima i nedostacima ovakvog pristupa razvoju softvera u poređenju sa savremenim editorima koji se oslanjaju na plugin ekosisteme.

1. Uvod

Projektovanje i implementacija tekstualnog editora predstavlja izazovan zadatak u oblasti softverskog inženjerstva. Razvoj ovakvog alata zahteva duboko razumevanje interakcija sa operativnim sistemom, upravljanje memorijom u realnom vremenu, efikasno manipulisanje strukturama podataka i dizajna korisničkog interfejsa unutar terminalskog okruženja.

Čak i kod minimalističkih rešenja potrebno je uskladiti niz kritičnih elemenata, kao što su niska latencija pri unosu, precizno renderovanje teksta, upravljanje kursorom i robusna obrada datoteka. Neusklađenost u bilo kom od ovih segmenata direktno narušava stabilnost alata i produktivnost korisnika.

Za razliku od tradicionalnih editora koji se oslanjaju na kompleksne konfiguracione fajlove, interpretirane skriptne jezike i dinamičke ekosisteme dodataka (engl. plugins), editor `rxw` usvaja radikalno drugačiji pristup. Editor se konfiguriše kroz izvorni kod primenom namenskih zakrpa (engl. patches). Izrada ovakvog softvera pruža odličnu platformu za demonstraciju tehnika sistemskog programiranja, optimizacije resursa i primene rigoroznih principa softverskog minimalizma inspirisanih suckless programima [2].

1.1. Opis problema

Savremeni inženjering softvera suočava se sa trendom konstantnog rasta složenosti i hardverske zahtevnosti razvojnih alata. Moderni editori koda i integrisana razvojna okruženja (engl. IDE) često troše stotine megabajta radne memorije i uvode kompleksne sisteme zavisnosti kroz procese instalacije dodataka (engl. plugins). Ovakav pristup ne samo da usporava rad na slabijim hardverskim konfiguracijama, već i značajno otežava kontrolu nad izvršnim kodom.

Iz tog razloga javlja se potreba za istraživanjem alternativnih i minimalističkih pristupa u razvoju softvera. Tema ovog rada je bitna jer se bavi optimizacijom performansi i povratkom korenima UNIX filozofije, gde softver treba da radi jednu stvar i da je radi kako treba [3]. Razvojem sopstvenog rešenja postiže se potpuna nezavisnost od ogromnih modernih editora i njihovih softverskih paketa, čime se dobija alat koji je u potpunosti prilagođen specifičnim navikama i radnom toku programera.

1.2. Ciljevi rada

Konkretni cilj ovog završnog rada jeste projektovanje i praktična realizacija terminal editora `rxw` koji kombinuje izuzetnu brzinu rada, minimalnu potrošnju resursa i visoku fleksibilnost.

Da bi se ovaj cilj ostvario, bilo je potrebno realizovati sledeće korake:

1. Analizirati arhitekture popularnih terminalskih editora i uočiti njihove prednosti i mane.
2. Implementirati editor u programskom jeziku C sa sledećim funkcionalnostima: rad u sirovom (engl. `raw`) režimu terminala, učitavanje i čuvanje fajlova, kretanje kursorom, uređivanje teksta, prikaz statusne linije, pretraga i zamena, `undo/redo`, `file browser`, isticanje sintakse određenih programskih jezika, modalno uređivanje, podrška za miš, bracketed paste, detekcija spoljašnje izmene fajla i kopiranje putem `OSC52 escape` sekvence.
3. Primeniti principe `suckless` filozofije i kreirati mehanizam konfiguracije zasnovan na zakrpama (engl. `patches`).
4. Razviti i testirati referentne zacrpe za proširene funkcionalnosti.
5. Izraditi pomoćne skripte koje korisniku omogućavaju lakše konfigurisanje, kompajliranje, instalaciju, uklanjanje i proširenje editora.
6. Izvršiti merenje performansi krajnjeg binarnog fajla i potrošnje memorije.
7. Pripremiti dokumentaciju projekta i objaviti izvorni kod na javnom git repozitorijumu [4].

2. Teorijske osnove

U ovom poglavlju prikazane su teorijske osnove značajne za razvoj terminalnog tekstualnog editora `rxw`. Obuhvaćeni su ključni aspekti sistemskog programiranja, specifičnosti programskog jezika C [5], manipulacija terminala kroz sirovi režim rada i kontrolne sekvence, kao i strukture podataka pogodne za efikasno upravljanje tekstualnim baferom.

Pored toga, detaljno su razmotreni principi suckless filozofije [2] softverskog inženjerstva, sa posebnim osvrtom na konfigurisanje i mehanizam kreiranja i primene zakrpa (engl. patches).

2.1. Analiza i komparativni prikaz postojećih terminalskih editora

U okviru ovog rada izvršena je detaljna analiza i komparacija srodnih terminalskih tekstualnih editora koji su direktno ili indirektno uticali na razvoj editora `rxw`.

Analizirani su sledeći sistemi:

- `vi` i `vim` - Industrijski standardi modalnog editovanja. Dok je izvorni `vi` postavio temelje efikasnog kretanja kroz tekst, `vim` (Vi IMproved) je uveo bogat ekosistem, napredno konfigurisanje kroz `.vimrc` fajlove i eksterni skripting jezik (Vimscript) [6].
- `neovim` - Moderan naslednik `vima` sa refaktorisanom bazom koda. Karakteriše ga asinhrona arhitektura, ugrađen LSP klijent i korišćenje Lua programskog jezika za konfiguraciju i razvoj dodataka (engl. plugins) [7].
- `nano` - Predstavnik jednostavnih, nemodalnih editora prilagođenih početnicima. Fokusiran je na brze izmene bez potrebe za učenjem kompleksnih prečica, ali nudi ograničene mogućnosti proširivanja i ograničene napredne opcije manipulacije teksta [8].
- `kilo` - Edukativni, minimalistički editor napisan u manje od 1000 linija C koda [1]. Iako funkcionalno limitiran, `kilo` demonstrira kako se manipulacijom terminala u sirovom (engl. raw) režimu može kreirati funkcionalan editor bez eksternih biblioteka.

U cilju jasnijeg sagledavanja arhitektonskih i filozofskih razlika između ovih alata i editora `rxw`, u tabelama ispod je dat uporedni pregled njihovih ključnih karakteristika.

Poređenje arhitektura terminalskih editora (Tabela 1).

Tabela 1. Arhitekture terminalskih editora

Editor	Arhitektura baze koda
rxw	Minimalistička monolitna (C)
kilo	Minimalistička (C fajl manji od 1000 LOC)
nano	Monolitna (C)
vi	Monolitna (istorijska C baza)
vim	Monolitna (kompleksna C baza)
neovim	Modularna, asinhrona (C i Lua)

Poređenje načina konfigurisanja terminalskih editora (Tabela 2).

Tabela 2. Način konfigurisanja

Editor	Način konfigurisanja
rxw	Zakrpama ili direktnom modifikacijom izvornog koda
kilo	Direktnom modifikacijom izvornog koda
nano	Eksterni fajl (.nanorc)
vi	Eksterni fajl (.exrc)
vim	Eksterni fajl (.vimrc), Vimscript
neovim	Eksterni fajlovi (Lua i Vimscript)

Poređenje sistema proširivanja terminalskih editora (Tabela 3).

Tabela 3. Sistem proširivanja

Editor	Sistemi proširivanja
rxw	Mehanizam zakrpa
kilo	Nema (ručne modifikacije izvornog koda)
nano	Nema
vi	Nema
vim	Dinamički plugin ekosistem (Vimscript)
neovim	Dinamički plugin ekosistem (Lua API)

Poređenje zavisnosti (engl. Dependencies) terminalskih editora (Tabela 4).

Tabela 4. Zavisi od

Editor	Zavisi od
rxw	Isključivo standardna C biblioteka
kilo	Isključivo standardna C biblioteka
nano	ncurses
vi	Standardne POSIX biblioteke
vim	Brojne eksterne biblioteke i parseri
neovim	Brojne eksterne biblioteke i parseri

Sa jedne strane imamo programe poput neovima koji teže maksimalnoj proširivosti po cenu kompleksnosti i veličini koda, dok s druge strane imamo edukativne alate poput kilo editora koji su ekstremno minimalni, ali nepraktični za svakodnevni rad. Razvoj editora rxw popunjava prazninu između ove dve krajnosti. Rxw zadržava nultu zavisnost od eksternih biblioteka i čistoću koda po uzoru na kilo, ali implementira robustan i čist mehanizam proširivanja inspirisan suckless filozofijom, omogućavajući korisniku da zadrži potpunu kontrolu nad funkcijama koje editor poseduje.

2.2. Suckless filozofija razvoja softvera

Tradicionalni razvoj modernog softvera često pati od fenomena poznatog kao software bloat, gde aplikacije vremenom postaju kompleksne, spore i pretrpane funkcionalnostima koje većina korisnika nikada ne koristi. Kao direktan odgovor na ovaj trend nastala je suckless filozofija [2], inženjerski pokret koji se zalaže za razvoj naprednog, ekstremno jednostavnog, jasnog i ekonomičnog softvera.

Glavni postulati suckless filozofije su:

- Jednostavnost i čitljivost - Kod mora biti napisan tako da ga jedan programer može u potpunosti razumeti i modifikovati.
- Minimalizam - Softver treba da radi samo jednu stvar i da je radi kako treba (UNIX filozofija) [3].
- Izbegavanje dinamičke kompleksnosti - Složeni grafički interfejsi i teški apstrakcioni slojevi se odbacuju u korist bazičnih, ali moćnih sistemskih mehanizama.

U kontekstu minimalnog editora, ovo znači da se ne uvode teške GUI biblioteke (engl. toolkit), ncurses biblioteka ili dinamičko učitavanje za dodatke (engl. plugins), već se maksimalno iskorišćavaju standardni POSIX mehanizmi [9] i standardni načini rada sa C programskim jezikom [5].

2.3. Konfigurisanje kroz izvorni kod naspram konfiguracionih fajlova

U klasičnim aplikacijama, prilagođavanje korisničkim opcijama se vrši u hodu, parsiranjem eksternih konfiguracionih fajlova, kao što su .json, .yaml, .ini ili namenskih skripti poput .vimrc. Iako je ovaj pristup naizgled komforan za korisnika, on sa sobom nosi ozbiljne arhitektonske nedostatke:

- Kompleksnost koda - Programer mora da napiše i održava podsistem za parsiranje teksta, validaciju sintakse i podsistem za rukovanje greškama u konfiguracionom fajlu.
- Potrošnja resursa - Aplikacija troši CPU cikluse i memoriju pri svakom pokretanju samo da bi učitala i interpretirala korisnička podešavanja.
- Skriveno ponašanje - Korisnik često ne zna tačno šta određeni plugin ili konfiguraciona opcija menja u toku rada.

Suckless softver eliminiše ovaj sloj tako što uvodi konfigurisanje preko zakrpa (engl. patches). Sva podešavanja (prečice na tastaturi, šeme boja, podrazumevana ponašanja) nalaze se unutar samog izvornog koda, najčešće u zasebnom zaglavlju (npr. config.h). Korisnik menja opcije direktno u kodu, a zatim ponovo kompajlira program. Sa obzirom na to da je kod čist i minimalan, proces kompajliranja traje manje od sekunde, dok izvršni fajl postaje maksimalno optimizovan jer kompajler tačno zna sve parametre unapred.

2.4. Mehanizam zakrpa (engl. patches) kao alternativa plugin ekosistemima

Da bi podržali proširivanje funkcionalnosti, standardni editori implementiraju kompleksne API sisteme i dinamička okruženja koja omogućavaju učitavanje eksternih dodataka (engl. plugins). Ovo otvara vrata bezbednosnim propustima, nestabilnosti i drastičnom usporavanju rada editora.

Kao alternativu dinamičkim dodacima (engl. plugins), suckless zajednica koristi mehanizam zakrpa (engl. patches) izvornog koda pomoću standardnih Unix alata diff i patch [3].

Koncept rada je sledeći, kada programer želi da doda novu funkcionalnost (npr. podršku za isticanje sintakse u editoru), on ne piše izolovani plugin. Umesto toga, on modifikuje postojeći izvorni kod i generiše .diff ili .patch fajl koji tačno mapira koje linije koda treba dodati, obrisati ili izmeniti.

Korisnik koji želi tu funkcionalnost, primenjuje zakrpu nad svojim izvornim kodom pomoću jednostavne komande

```
patch < funkcionalnost.diff
```

Nakon uspešnog korišćenja mehanizma zakrpa (engl. patches), softver se ponovo kompajlira. Prednosti ovog pristupa u razvoju editora rwx su višestruke:

- Nulta latencija - Nova funkcionalnost postaje integralni deo binarnog fajla nakon kompajliranja, nema dinamičkog učitavanja, niti interpretiranja u realnom vremenu.
- Potpuna kontrola - Korisnik u svakom trenutku ima uvid u to šta se tačno menja u izvornom kodu, čime se eliminiše rizik od skrivenih grešaka ili zlonamernog koda.
- Modularnost bez neželjenih efekata - Editor ostaje savršeno prilagođen potrebama pojedinca i sadrži isključivo one linije koda koje su korisniku zaista potrebne.

3. Rešavanje problema

U ovom poglavlju detaljno je opisan proces projektovanja i implementacije terminalnog editora `rxw`. Implementacija je izvedena u programskom jeziku C, čime se obezbeđuje visoka portabilnost softvera na različitim operativnim sistemima.

Prilikom pokretanja, program vrši manipulaciju terminala kako bi omogućio prelazak u sirovi (engl. raw) režim rada, čime se preuzima direktna kontrola nad svakim unetim karakterom i iscrtavanjem ekrana.

3.1. Arhitektura i organizacija koda

3.1.1. Opšta objašnjenja

Editor `rxw` implementiran je u jednom C fajlu (`rxw.c`), što je u skladu sa suckless pristupom koji preferira mali, čitljivi i lako promenljivi kod, takođe kompajleri malo bolje optimizuju konačni binarni fajl ako je sve u jednom C fajlu.

Fajl je podeljen na logične sekcije:

- defines (konstante, boje, tipovi isticanja (engl. highlight), akcije)
- Includes (pozivanje potrebnih biblioteka)
- file types and keywords (tipovi fajlova i njihove ključne reči)
- data (strukture podataka)
- prototypes (prototipovi funkcija)
- helper (pomoćne funkcije)
- memory arena (arena alokatori memorije)
- init (inicijalizacija editora).
- terminal (pripremanje terminala)
- syntax highlighting (Isticanje sintakse).
- row operations (operacije nad redovima).
- editor operations (editorske operacije).
- file browser (pregledač fajlova)
- file I/O (operacije nad fajlom)
- find (pretraga)
- input (operacije nad ulazima (miš i tastatura))
- output (operacije nad izlazom (monitor)).

Glavna struktura editora je struct config E, koja čuva globalno stanje, u kome se nalazi: naziv fajla, niz redova, trenutni kursor, offsete prikaza, undo/redo stakove, stanje izbora, režim editovanja, statusnu poruku i ostale parametre.

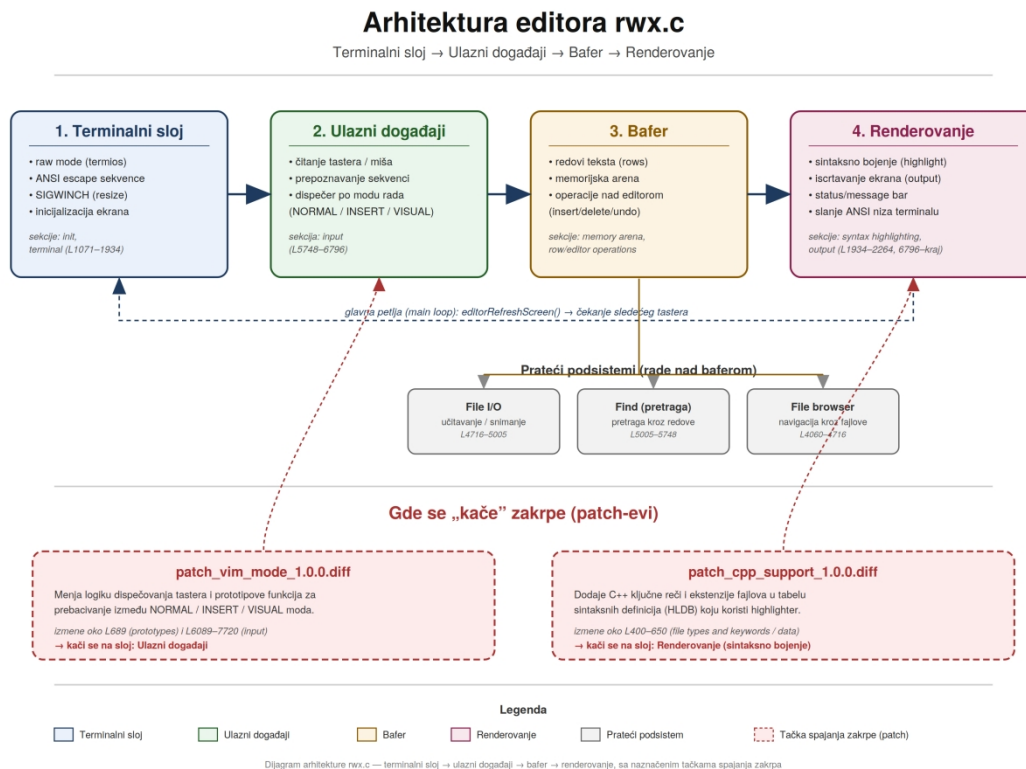
Glavna petlja programa je jednostavna i jasna.

```
while(1)
{
    if(window_resized)
    {
        window_resized=0;
        get_window_size(&E.term_rows,&E.screen_cols);
    }
    check_file_modified_externally();
    refresh_screen();
    process_keypress();
}
```

Ova petlja kontinuirano:

- proverava da li je terminal promenjen (engl. resize),
- proverava da li je fajl izmenjen spolja,
- crta ekran,
- čeka i obrađuje sledeći korisnički ulaz.

Prikaz arhitekture rxw editora (Slika 1).



Slika 1. Arhitektura rxw editora

3.2. Terminalni sloj i sirovi režim rada

Ključna karakteristika terminal editora je prelazak terminala u sirov (engl. raw) mode, gde operativni sistem ne obrađuje ulaz na uobičajen način, već se svaki pritisak na tastaturi i mišu direktno prosleđuje aplikaciji.

U POSIX okruženju koristi se termios struktura [5]

```
struct termios raw=E.original_termios;  
raw.c_iflag &= ~(BRKINT|ICRNL|INPCK|ISTRIP|IXON);  
raw.c_oflag &= ~(OPOST);  
raw.c_lflag &= ~(ECHO|ICANON|IEXTEN|ISIG);  
raw.c_cc[VMIN]=0;  
raw.c_cc[VTIME]=1;
```

Ovim kodom se isključuju:

- ECHO - automatsko ispisivanje karaktera,
- ICANON - kanonska linija (Enter završava liniju),
- ISIG - signal na Ctrl-C,
- OPOST - post-processing izlaza,
- IXON - pause/resume kontrola toka.

Dok u Windows okruženju se koristi SetConsoleMode

- ENABLE_ECHO_INPUT,
- ENABLE_LINE_INPUT,
- ENABLE_PROCESSED_INPUT,
- ENABLE_QUICK_EDIT_MODE,
- ENABLE_VIRTUAL_TERMINAL_PROCESSING.

Editor koristi sledeće ANSI/terminalne escape sekvence:

- \x1b[?25l - sakrij kursor,
- \x1b[?25h - pokaži kursor,
- \x1b[H - kursor na početak ekrana,
- \x1b[2J - očisti ekran,
- \x1b[K - očisti ostatak linije,
- \x1b[...m - SGR boje i stilovi [9],
- \x1b[?1002h - X10 mouse tracking
- \x1b[?1006h - SGR mouse tracking,
- \x1b[?2004h - bracketed paste.

Funkcija `read_key()` čita ulaz i dekoduje escape sekvence u interne vrednosti kao što su `ARROW_UP`, `HOME_KEY`, `MOUSE_LEFT_PRESS`, `BRACKETED_PASTE`. Ovo omogućava da ostatak editora ne mora da zna detalje terminalnih sekvenci.

Za promenu veličine terminala na POSIX sistemima koristi se signal `SIGWINCH`, dok se na Windows sistemima dešavanja čitaju kroz `ReadConsoleInputW`.

3.3. Struktura podataka i upravljanje memorijom

Svaka linija teksta u editoru čuvana je u strukturi `e_row`:

```
typedef struct
{
    char* characters;
    char* render;
    uint8_t* highlight;
    int32_t* wrap_starts;
    int32_t id;
    int32_t size;
    int32_t r_size;
    int32_t gap_start;
    int32_t gap_end;
    int32_t buf_size;
    int32_t wrap_seg_count;
    int32_t wrap_starts_cap;
    int32_t wrap_epoch;
    uint8_t h_open_comment;
}e_row;
```

Polja imaju sledeće značenje:

- `characters` - originalni tekst reda (bafer za uređivanje),
- `render` - renderovani prikaz reda (tabovi, kontrolni i hex karakteri),
- `highlight` - niz `highlight` oznaka za renderovane karaktere,
- `wrap_starts` - bafer za početke prelomljenih segmenata,
- `id` - identifikator reda,
- `size` - dužina originalnog teksta,
- `r_size` - dužina renderovanog prikaza,
- `gap_start`, `gap_end` - granice `gap` bafera za efikasno uređivanje,
- `buf_size` - ukupna dodeljena veličina `characters` bafera,
- `wrap_seg_count` - broj segmenata prelamanja,
- `wrap_starts_cap` - kapacitet `wrap_starts` bafera,
- `wrap_epoch` - keš za prelamanje reda,
- `h_open_comment` - zastava koja nam govori da li je komentar otvoren na ovom redu.

Za efikasno uređivanje reda koristi se gap buffer. Insertovanje i brisanje u trenutnoj poziciji vrši se pomeranjem granica bafera pomoću promenljiva gap_start i gap_end, bez pomeranja celog teksta.

Ključna funkcija za pomeranje gap bufera je:

```
static void row_gap_move_to(e_row* row,int32_t pos)
{
    if(pos == row->gap_start)
    {
        return;
    }
    if(pos < row->gap_start)
    {
        int32_t n=row->gap_start-pos;
        memmove(&row->characters[row->gap_end-n],&row->characters[pos],n);
        row->gap_start-=n;
        row->gap_end-=n;
    }
    else
    {
        int32_t n=pos-row->gap_start;
        memmove(&row->characters[row->gap_start],&row->characters[row->gap_end],n);
        row->gap_start+=n;
        row->gap_end+=n;
    }
}
```

Ova tehnika je pogodna za editor koji često uređuje tekst na trenutnoj poziciji kursora.

Pored gap bafera, editor koristi i memorijsku arenu za privremene baferove renderovanja i highlighta:

```
mem_arena* arena_create(uint64_t reserve_size,uint64_t commit_size);
void arena_destroy(mem_arena* arena);
void* arena_push(mem_arena* arena,uint64_t size,uint8_t zero);
void arena_pop(mem_arena* arena,uint64_t size);
void arena_pop_to(mem_arena* arena,uint64_t pos);
void arena_clear(mem_arena* arena);
static void compact_row_arenas();
```

Arena rezerviše veliki virtualni memorijski opseg, ali zauzima (engl. commit) samo onoliko koliko je trenutno potrebno fizičke memorije.

U init_editor() definisano je:

```
render_arena=arena_create(MB(256),KB(64));
highlight_arena=arena_create(MB(128),KB(64));
```

Time editor može da radi i sa većim fajlovima bez da odmah zauzme velike cele fizičke memorijske opsege.

Neki od osnovnih parametara rwx editora (Tabela 5).

Tabela 5. Osnovni parametri rwx editora

Parametar	Vrednost	Opis
SIZE_LIMIT	1024	Maksimalan broj undo/redo akcija
ROW_ARENA_COMPACT_THRESHOLD	128 KB	Prag za kompaktno čišćenje arena
Render arena reserve	256 MB	Virtualna rezervacija za render bafer
Highlight arena reserve	128 MB	Virtualna rezervacija za highlight bafer
Početni commit	64 KB	Početna fizička memorija arene
Chunk čitanja fajla	4096 bajtova	Veličina čitanja pri učitavanju

3.4. Učitavanje, čuvanje i detekcija eksterne izmene fajla

Funkcija `open_file()` učitava fajl u niz redova. Čitanje se obavlja u komadima (engl. chunk) od 4096 bajtova, čime se izbegava alociranje celog fajla odjednom u baferu.

Tokom učitavanja:

- novi redovi se dodaju kroz `insert_row()`,
- `\r` se uklanja kako bi se ispravno obradili Windows fajlovi,
- ako fajl ne postoji, otvara se novi fajl,
- nakon učitavanja poziva se `select_syntax_highlight()` da bi se odredila sintaksa po ekstenziji.

Funkcija `save_file()` prvo formira celokupan sadržaj fajla u `rows_to_string()`, a zatim otvara fajl, odseca ga na zadanu veličinu i upisuje podatke.

```
file_desc=open(E.file_name,O_RDWR|O_CREAT,0644);  
ftruncate(file_desc,len);  
write(file_desc,buffer,len);
```

Ako fajl nije sačuvan, u statusnoj liniji se prikazuje poruka sa greškom.

Detekcija eksterne izmene vrši se funkcijom `check_file_modified_externally()` koja upoređuje trenutno vreme izmene fajla (`mtime`) sa poslednjim poznatim vremenom izmene.

Ako je fajl menjan izvan editora, prikazuje se upozorenje:

```
WARNING: File was modified by another program (Ctrl+s to overwrite, Ctrl+p to reload)
```

Korisnik može:

- `Ctrl+s` - prepisati (engl. overwrite) fajl,
- `Ctrl+p` - ponovo učitati sa diska.

3.5. Modalno uređivanje i obrada korisničkog ulaza

Editor poseduje dva osnovna modalna stanja:

- MODE_NORMAL - normalni režim
- MODE_INSERT - insert režim.

Prelazak iz jednog u drugo vrši se preko Ctrl+i.

U normalnom režimu, tasteri W,A,S,D koriste se za kretanje, dok strelice (engl. arrow keys) rade u svakom modu. Ovo je po uzoru na modalne editore, ali uz dodatne W/A/S/D prečice koje pojednostavljaju kretanje.

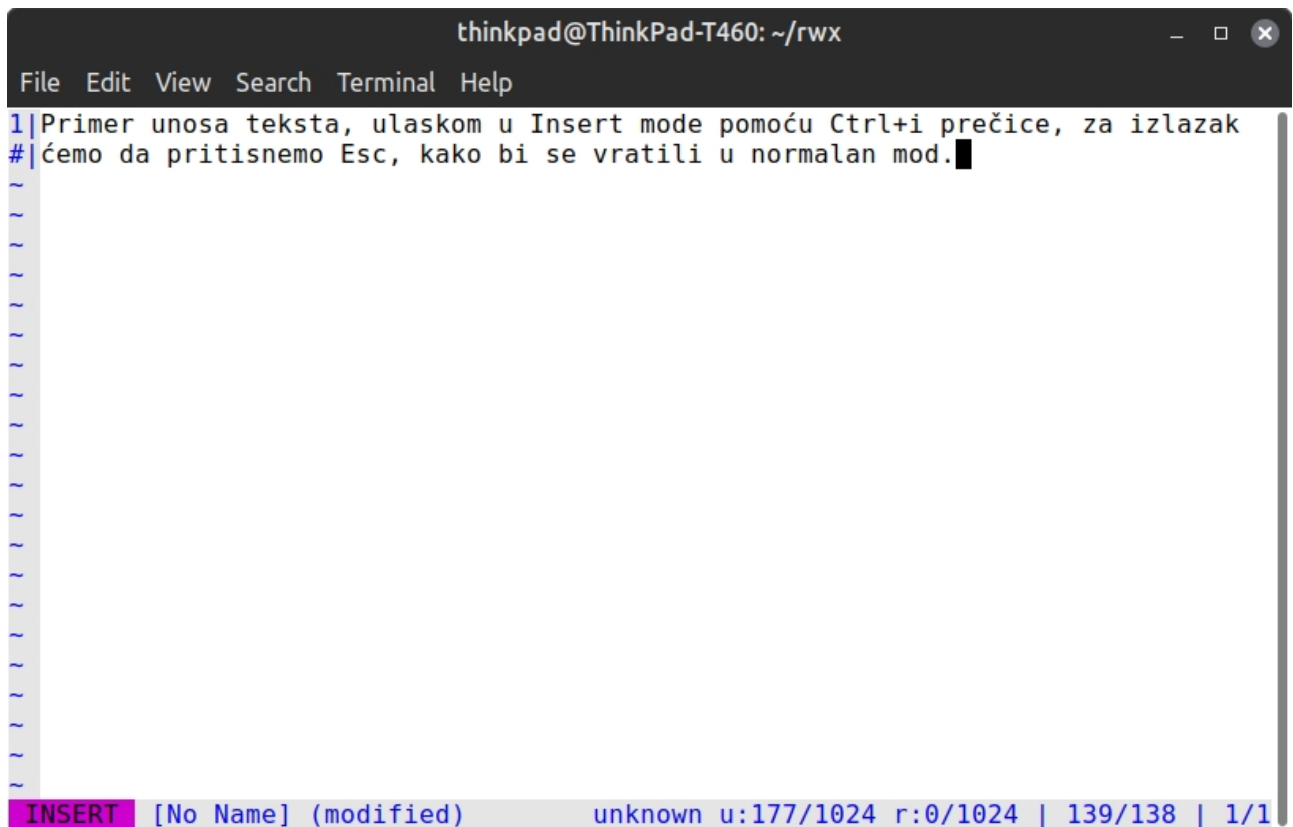
U insert režimu, pisanje karaktera direktno menja sadržaj reda. Pre svake izmene, editor formira actions strukturu i dodaje je na undo stak, čime se omogućava kasniji povratak na prethodno stanje.

Prečice rwx editora (Tabela 6).

Tabela 6. Prečice rwx editora

Prečica	Funkcija
Ctrl+s	Sačuvaj fajl (engl. save)
Ctrl+q	Izlaz (2 puta ako ima nesačuvanih promena)
Ctrl+i	Prebacivanje u insert/normal režim
Ctrl+e	Aktiviranje/isključivanje označavanja
Ctrl+z	Opozovi radnju (engl. undo)
Ctrl+r	Ponovo uradi radnju (engl. redo)
Ctrl+o	Otvori pretraživač fajlova (engl. file browser)
Ctrl+p	Ponovo učitaj fajl
Ctrl+g	Idi na red broj
Ctrl+x	Kopiraj i obriši (engl. cut)
Ctrl+c	Kopiraj (engl. copy)
Ctrl+v	Ubaci kopirani tekst (engl. paste)
Ctrl+f	Pretraga (engl. find)
Ctrl+w	Pretraga i zamena
Ctrl+h	Pomoć (engl. help)
ESC	Izlaz iz insert režima ili date funkcije
Home/End	Početak/kraj reda
Page Up/Page Down	Prethodna/sledeća strana
Miš	Kretanje, označavanje, dvoklik za označavanje reči, wheel za scroll i desni klik za kopiranje

Primer unosa teksta, ulaskom u Insert mode pomoću Ctrl+i prečice, za izlazak iz Insert moda koristimo Esc ili Ctrl+i još jednom, kako bi se vratili u normalan mod (Slika 3).



Slika 3. Insert mode

3.6. Pretraga, zamena i navigacija

Pretraga je implementirana pomoću sledećih funkcija:

- `find()`
- `find_callback()`
- `find_last_in_row_before()`
- `find_next_match()`
- `count_matches()`

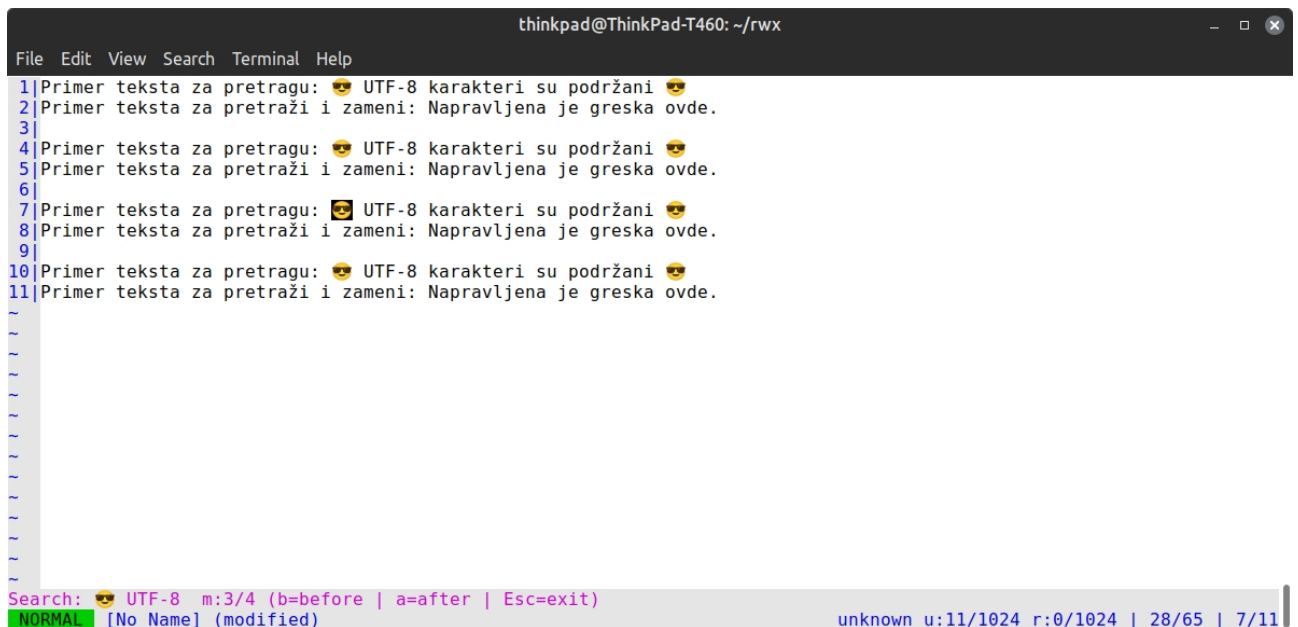
Pretraga se pokreće pomoću prečice Ctrl+f.

Editor prikazuje prompt:

Search: (Use Esc/Arrows/Enter)

Tokom kucanja upita, callback `find_callback()` ažurira statusnu poruku i poziciju kursora na sledeću ili prethodnu reč koja se podudara (engl. match).

Kada pritisnemo Enter dobijamo opcije (b i a), pomoću kojih možemo da odaberemo na kojoj reči ćemo da se zaustavimo (Slika 5).



Slika 5. Prečice za pretragu

Pomoću prečice Ctrl+w pretražujemo fajl za reči koje se podudaraju kako bi smo ih kasnije mogli zameniti (Slika 6).



Slika 6. Pretraži deo opcije pretraži i zameni

Kada pritisnemo Enter dobijamo sledeće polje, pomoću kojeg unosimo reč sa kojom ćemo da zamenimo prethodnu reč (Slika 7).

A screenshot of a terminal window on a ThinkPad T460. The terminal title is "thinkpad@ThinkPad-T460: ~/rwx". The menu bar shows "File", "Edit", "View", "Search", "Terminal", and "Help". The terminal content shows a search and replace operation. The search results are listed as follows:
1|Primer teksta za pretragu: UTF-8 karakteri su podržani
2|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
3|
4|Primer teksta za pretragu: UTF-8 karakteri su podržani
5|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
6|
7|Primer teksta za pretragu: UTF-8 karakteri su podržani
8|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
9|
10|Primer teksta za pretragu: UTF-8 karakteri su podržani
11|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
Below the search results, there is a search bar with the text "Search: UTF-8 karakteri su podržani" and a replace bar with the text "Replace: Tekst koji menja prethodni tekst.". At the bottom of the terminal, there is a status bar showing "NORMAL [No Name] (modified)" and "unknown u:11/1024 r:0/1024 | 28/65 | 1/11".

Slika 7. Unos reči kojom menjamo prethodnu

Kada pritisnemo Enter dobijamo opcije (b, a, r i d), pomoću kojih možemo da odaberemo koje reči hoćemo da zamenimo (Slika 8).

```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
1|Primer teksta za pretragu: UTF-8 karakteri su podržani 
2|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
3|
4|Primer teksta za pretragu: UTF-8 karakteri su podržani 
5|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
6|
7|Primer teksta za pretragu: UTF-8 karakteri su podržani 
8|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
9|
10|Primer teksta za pretragu: UTF-8 karakteri su podržani 
11|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
~
~
~
~
~
~
~
~
Search: UTF-8 karakteri su podržani 
Replace: Tekst koji menja prethodni tekst. m:2/4 (b=before | a=after | r=replace | d=replace all | Esc=exit)
NORMAL [No Name] (modified) unknown u:11/1024 r:0/1024 | 28/65 | 4/11
```

Slika 8. Prečice za zamenu

Kada pritisnemo opciju **r** menjamo datu reč (Slika 9).

```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
1|Primer teksta za pretragu: 🤖 UTF-8 karakteri su podržani 🤖
2|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
3|
4|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
5|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
6|
7|Primer teksta za pretragu: 🤖 UTF-8 karakteri su podržani 🤖
8|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
9|
10|Primer teksta za pretragu: 🤖 UTF-8 karakteri su podržani 🤖
11|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
Search: 🤖 UTF-8 karakteri su podržani 🤖
Replace: Tekst koji menja prethodni tekst. m:2/3 Replaced 1 occurrence (b=before | a=after | r=replace | d=replace all | Esc=exit)
NORMAL [No Name] (modified) unknown u:13/1024 r:0/1024 | 28/65 | 7/1
```

Slika 9. Menjanje reči

Kada pritisnemo opciju d menjamo sve reči koje se podudaraju (Slika 10).

```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
1|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
2|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
3|
4|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
5|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
6|
7|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
8|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
9|
10|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
11|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
Search: UTF-8 karakteri su podržani
Replace: Tekst koji menja prethodni tekst. m:0/0 Replaced 3 occurrence(s) (b=before | a=after | r=replace | d=replace all | Esc=exit)
NORMAL [No Name] (modified) unknown u:19/1024 r:0/1024 | 61/60 | 10/11
```

Slika 10. Menjanje svih reči

Pomoću prečice Ctrl+g tražimo i odlazimo na dati red (Slika 11).

```
thinkpad@ThinkPad-T460: ~/rwx
```

	File	Edit	View	Search	Terminal	Help
1	Primer teksta za pretragu:	Tekst koji menja prethodni tekst.				
2	Primer teksta za pretraži i zameni:	Napravljena je greska ovde.				
3						
4	Primer teksta za pretragu:	Tekst koji menja prethodni tekst.				
5	Primer teksta za pretraži i zameni:	Napravljena je greska ovde.				
6						
7	Primer teksta za pretragu:	Tekst koji menja prethodni tekst.				
8	Primer teksta za pretraži i zameni:	Napravljena je greska ovde.				
9						
10	Primer teksta za pretragu:	Tekst koji menja prethodni tekst.				
11	Primer teksta za pretraži i zameni:	Napravljena je greska ovde. █				
~						
~						
~						
~						
~						
~						
~						
~						
~						
~						
~						
Go to line: 5						
NORMAL	[No Name]	(modified)				
					unknown u:19/1024 r:0/1024 65/64 11/11	

Slika 11. Idi na red

Data naredba je izvršena i nalazimo se na redu broj 5 (Slika 12).

```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
1|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
2|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
3|
4|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
5|Primer teksta za pretraži i zameni: Napravljena je greska ovde. █
6|
7|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
8|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
9|
10|Primer teksta za pretragu: Tekst koji menja prethodni tekst.
11|Primer teksta za pretraži i zameni: Napravljena je greska ovde.
~
~
~
~
~
~
~
~
~
~
NORMAL [No Name] (modified) unknown u:19/1024 r:0/1024 | 65/64 | 5/11
```

Slika 12. Na želenom redu smo

3.7. Isticanje sintakse i renderovanje teksta

Isticanje sintakse realizovano je kroz strukturu syntax:

```
typedef struct
{
    const char* file_type;
    const char* const* file_ext;
    const char* const* keywords;
    const uint8_t* keywords_len;
    const uint8_t* keywords_syntax_color;
    const char* single_line_comment_start;
    const char* multi_line_comment_start;
    const char* multi_line_comment_end;
}syntax;
```

U trenutnoj verziji definisan je samo jedan programski jezik u bazi za sintakse i taj jezik je C programski jezik:

```
syntax H_DB[]=
{
    {"C",C_H_extensions,C_H_keywords,C_H_keywords_len,C_H_keywords_syntax_color,"//","/*","*/"}
};
```

Podržane ekstenzije su:

- .c
- .h
- .i

Highlight se vrši po renderovanom prikazu linije, a ne direktno po originalnim karakterima, jer se tabovi, kontrolni i hex karakteri pri renderovanju šire.

Tipovi highlighta su:

- H_NORMAL
- H_COMMENT
- H_M_COMMENT
- H_KEYWORD_1
- H_KEYWORD_2
- H_KEYWORD_3
- H_STRING
- H_NUMBER
- H_MATCH
- H_CTRL
- H_HEX

Ključne reči su organizovane sa bojama:

- H_KEYWORD_1 (preprocessorske ključne reči)
- H_KEYWORD_2 (npr. tipovi podataka i posebne ključne reči)
- H_KEYWORD_3 (ostale ključne reči)

Funkcija `update_syntax()`:

- dodeljuje novi highlight bafer iz `highlight_arena`,
- inicijalizuje sve karaktere kao `H_NORMAL`,
- skenira render tekst,
- prepoznaje jedno linijske komentare,
- prepoznaje više linijske komentare,
- prepoznaje stringove,
- prepoznaje brojeve,
- prepoznaje ključne reči.

Višelinijski komentari se prate pomoću zastave `h_open_comment`.

Renderovanje ekrana vrši se funkcijom `draw_rows()`, ona:

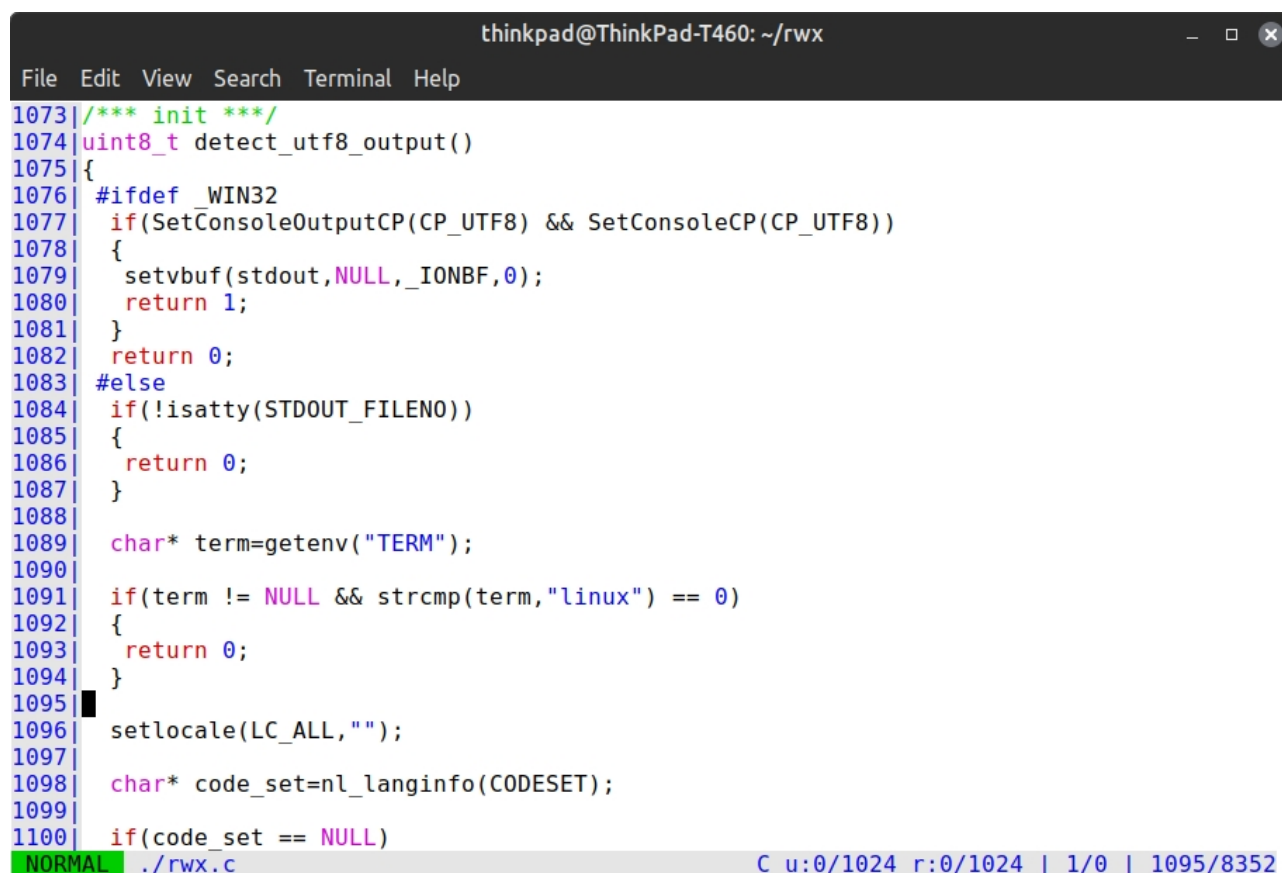
- crta brojeve linija,
- crta renderovani tekst,
- menja boje prema `highlightu`,
- ističe izbor invertovanjem boja,
- obrađuje tab karaktere,
- prikazuje kontrolne karaktere kao `Ctrl+X`,
- prikazuje karaktere koji nisu validni UTF-8 karakteri kao `<xx>`,
- crta statusnu liniju,
- pozicionira kursor.

Za dugi tekst postoje funkcije za prelamanje linija (engl. `wrapping`), funkcije koje su zadužene za to su:

- `row_should_wrap()`
- `get_wrap_segments()`
- `count_screen_rows()`
- `get_cursor_segment_info()`

Ove funkcije određuju na koje segmente se linija teksta lomi i kako se pozicionira kursor u višerednom prikazu.

Prikaz rwx editora sa isticanjem sintakse (Slika 13).



```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
1073|/** init **/
1074|uint8_t detect_utf8_output()
1075|{
1076|#ifdef WIN32
1077|    if(SetConsoleOutputCP(CP_UTF8) && SetConsoleCP(CP_UTF8))
1078|    {
1079|        setvbuf(stdout,NULL,_IONBF,0);
1080|        return 1;
1081|    }
1082|    return 0;
1083|#else
1084|    if(!isatty(STDOUT_FILENO))
1085|    {
1086|        return 0;
1087|    }
1088|
1089|    char* term=getenv("TERM");
1090|
1091|    if(term != NULL && strcmp(term,"linux") == 0)
1092|    {
1093|        return 0;
1094|    }
1095|
1096|    setlocale(LC_ALL,"");
1097|
1098|    char* code_set=nl_langinfo(CODESET);
1099|
1100|    if(code_set == NULL)
NORMAL ./rwx.c C u:0/1024 r:0/1024 | 1/0 | 1095/8352
```

Slika 13. Isticanje sintakse

3.8. Pretraživač fajlova

Kada se editor pokrene sa direktorijumom kao argumentom ili kada korisnik pritisne Ctrl+o, otvara se pretraživač fajlova (engl. file browser).

Pretraživač fajlova (engl. file browser):

- lista fajlove i direktorijume,
- podržava prikazivanje skrivenih fajlova,
- podržava dugi format prikaza ako je terminal dovoljno širok,
- omogućava navigaciju strelicama, mišem i slovima W/A/S/D,
- otvara direktorijum na Enter ili dvoklikom miša,
- ide nazad na Backspace,
- izlazak iz pretraživača na ESC ili q.

Struktura ulaza je browse_entry:

```
typedef struct
{
    char name[256];
    uint8_t is_dir;
}browse_entry;
```

Sortiranje se vrši alfabetski pomoću qsort() i funkcije browse_compare_entries().

Na POSIX sistemima koristi se opendir/readdir/stat, dok se na Windows sistemima koristi FindFirstFileA/FindNextFileA.

U dugom formatu prikazuju se:

- permisije,
- broj hard linkova,
- owner,
- grupa,
- veličina,
- datum izmene,
- naziv fajla.

Prikaz pretraživača fajlova (Slika 14).



Slika 14. Pretraživač fajlova

3.9. Undo/redo i izbor teksta

Undo/redo je implementiran pomoću dva staka veličine SIZE_LIMIT (1024).

Svaka korisnička akcija se čuva u strukturi actions:

```
typedef struct
{
    char* row_text;
    uint32_t row_len;
    int32_t c_x;
    int32_t c_y;
    int32_t c_x_end;
    int32_t c_y_end;
    int32_t join_col;
    uint8_t type;
    uint8_t c_len;
    char c[6];
}actions;
```

Tipovi akcija su:

- USER_ACTION_INSERT_CHAR
- USER_ACTION_DEL_CHAR
- USER_ACTION_INSERT_NEWLINE
- USER_ACTION_JOIN_LINES
- USER_ACTION_ROW_INSERT
- USER_ACTION_ROW_DELETE
- USER_ACTION_RANGE_INSERT
- USER_ACTION_RANGE_DELETE

Kada se izvrši undo, akcija prelazi u redo stak.

Kada korisnik izvrši novu akciju, redo stak se briše.

Editor takođe ima izbor teksta:

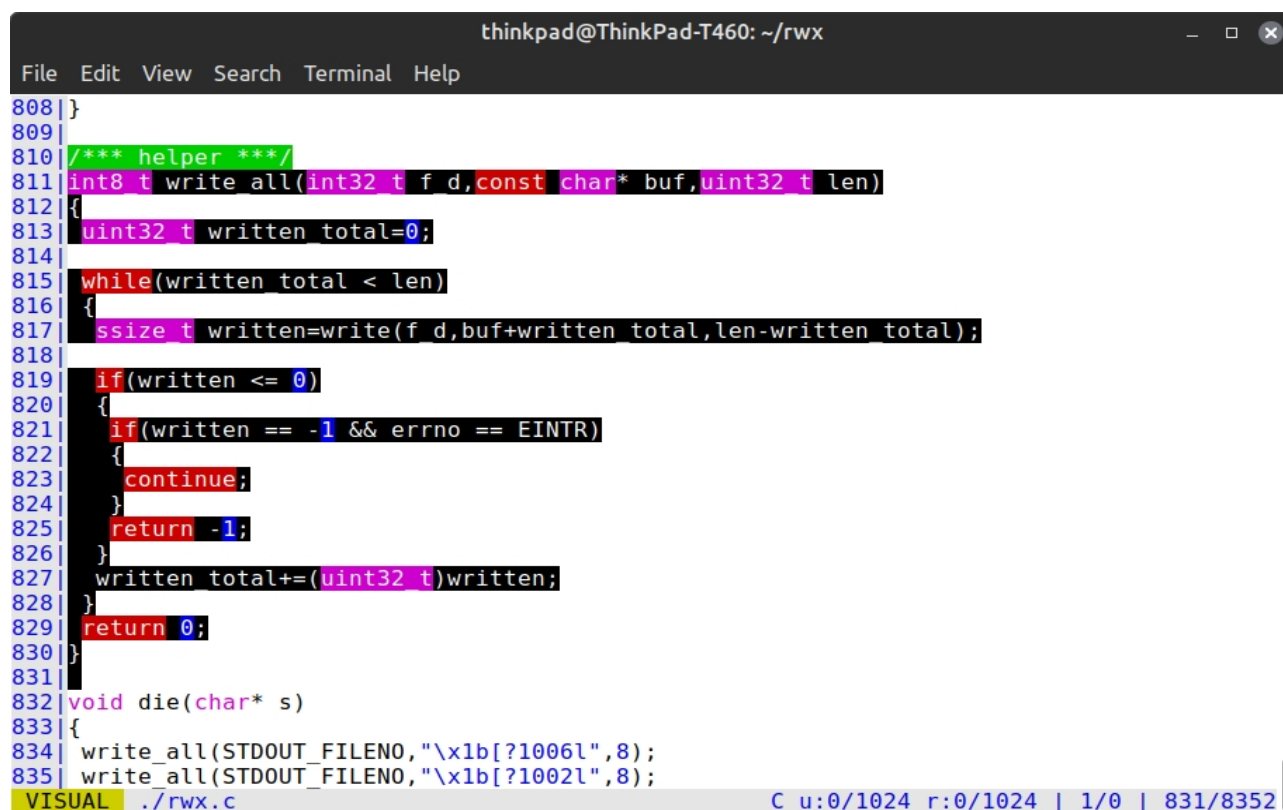
- SELECT_NONE
- SELECT_CHAR
- SELECT_ROW

Izbor teksta se može aktivirati pomoću:

- Ctrl+e
- mišem
- double click za izbor reči

Kopiranje se vrši preko OSC52 escape sekvence, što omogućava kopiranje u terminalnom clipboardu bez potrebe za eksternim alatima.

Prikaz izbora teksta pomoću prečice Ctrl+e (Slika 15).



```
thinkpad@ThinkPad-T460: ~/rwx
File Edit View Search Terminal Help
808 }
809
810 /** helper **/
811 int8_t write_all(int32_t f_d, const char* buf, uint32_t len)
812 {
813     uint32_t written_total=0;
814
815     while(written_total < len)
816     {
817         ssize_t written=write(f_d,buf+written_total,len-written_total);
818
819         if(written <= 0)
820         {
821             if(written == -1 && errno == EINTR)
822             {
823                 continue;
824             }
825             return -1;
826         }
827         written_total+=(uint32_t)written;
828     }
829     return 0;
830 }
831
832 void die(char* s)
833 {
834     write_all(STDOUT_FILENO, "\x1b[?1006l", 8);
835     write_all(STDOUT_FILENO, "\x1b[?1002l", 8);
836 }
VISUAL ./rwx.c C u:0/1024 r:0/1024 | 1/0 | 831/8352
```

Slika 15. Izbor teksta

3.10. Merenje performansi

Merenja su izvršena na x86_64 Linux Mint mašini sa procesorom Ryzen 9 5950x i sa 64 GB RAM memorije.

Windows 11 testiranje je obavljeno u virtualnoj mašini na istom računaru gde je virtuelnoj mašini dato 8 jezgara i 8 niti procesorskih resursa i 8 GB RAM memorije. Kompajliranje je obavljeno sa gcc kompajlerom i sa zastavom -O2 na oba operativna sistema.

Na Linux Mint sistemu veličina binarnog fajla je 113kB dok je na Windows 11 sistemu 165 kB.

Za ovo merenje performansi merena je samo potrošnja RAM memorije jer merenje korišćenosti procesorskih resursa nije baš precizno niti lako za uočiti sa ovako malim programima. Program sa kojim sam merio potrošnju RAM memorije na Linux Mint sistemu je System Monitor (program je sličan Windows Task Manager programu) dok na Windows 11 operativnom sistemu koristio sam Windows Task Manager da izmerim potrošnju RAM memorije.

Editor sa kojim sam uporedio rwx editor je neovim, jer je on jedini editor koji većina korisnika koristi za svakodnevni rad i da pri tome radi na većini platformi.

Testiralo se prvo korišćenje RAM memorije prilikom otvaranja samog editora bez otvaranja fajla, pa kao drugi test se uzelo korišćenje RAM memorije prilikom otvaranja `cpp_syntax_patch.c` fajla, dužine 143 linije C koda. Rezultate merenja možemo da vidimo u sledećoj tabeli (Tabela 7).

Tabela 7. Merenja performansi

Test	RWX	NEOVIM
Otvaranje samog editora (Linux)	98,3kB	6,5MB
Otvaranje samog editora (Windows)	0,5MB	7,7MB
Otvaranje <code>cpp_syntax_patch.c</code> fajla (Linux)	192,5kB	6,8MB
Otvaranje <code>cpp_syntax_patch.c</code> fajla (Windows)	0,6MB	8,8MB

Terminalni programi uglavnom bolje rade na Linux i drugim POSIX operativnim sistemima u poređenju sa Windows operativnim sistemom. Kao što vidimo, što se tiče korišćenja RAM memorije rwx je u kB dok je neovim u MB. Time možemo da zaključimo da je rwx editor znatno bolji što se tiče korišćenja memorije.

3.11. Dokumentacija i objavljivanje koda

Izvorni kod editora objavljen je na javnom Git repozitorijumu [6]: <https://github.com/ZeljkoRasovic/rwx>

U repozitorijumu su:

- `rwx.c`
- README uputstvom
- primeri referentnih zakrpa (engl. patches)
- skripte `configure`, `compile`, `install`, `uninstall`, `patch_code`
- testni fajlovi za sintaksu

Dve referentne zakrpe koje su izrađene kao primeri za korisnike:

- Zakrpa za isticanje C++ sintakse - dodaje bazu sintakse i highlight za C++ fajlove.
- Zakrpa za vim prečice - dodaje slične prečice kao vi i vim.

3.12. Pomoćne skripte za razvoj, instalaciju i održavanje

U okviru projekta izrađene su skripte koje korisniku omogućavaju da izvrši sve osnovne operacije nad editorom bez ručnog pokretanja terminalnih komandi. Skripte postoje u dve varijante: shell skripte (`*.sh`) za POSIX sisteme i batch skripte (`*.bat`) za Windows sisteme. Obe varijante imaju istu logiku i interfejs, pa korisnik na bilo kom operativnom sistemu dobija isto iskustvo.

3.12.1. Skripta configure

Skripta configure.sh (odnosno configure.bat) služi za detekciju dostupnog C kompajlera i generisanje fajla config.txt koji se kasnije koristi od strane skripti compile, install i uninstall.

Tokom izvršavanja, skripta:

1. proverava da li je promenljiva okruženja CC postavljena i da li taj kompajler postoji,
2. redom proverava prisustvo gcc, clang i cc (u .bat verziji je i cl (MSVC) podržan)
3. bira prvi nađen kompajler

Generiše config.txt sa sledećim poljima:

```
CC="<detektovani_kompajler>"
CFLAGS="<podrazumevani_flagovi>"
LDFLAGS="<linker_flagovi>"
BIN_EXT="<ekstenzija_binarnog_fajla>"
DESTDIR="<ciljni_direktorijum>"
PREFIX="<prefiks_instalacije>"
BINDIR="<direktorijum_binarnih_fajlova>"
INSTALL_MSG="<poruka_pri_instalaciji>"
UNINSTALL_MSG="<poruka_pri_uklanjanju>"
INSTALL_CMD="<komanda_za_instalaciju>"
UNINSTALL_CMD="<komanda_za_uklanjanje>"
```

Na POSIX sistemima, podrazumevani PREFIX je /usr/local, a BIN_EXT je prazan string, dok je na Windows sistemima PREFIX je C:\Program Files\rwx, a BIN_EXT je .exe.

Primer korišćenja:

Posix (Linux, MacOS, FreeBSD, OpenBSD, NetBSD):

```
sh configure.sh
```

Windows:

```
configure.bat
```

3.12.2. Skripta compile

Skripta compile.sh (odnosno compile.bat) kompajlira zadati C fajl koristeći postavke iz config.txt ili iz okruženja. Podržava sledeće pozive:

```
sh compile.sh [kompajler] [flagovi] <file.c>
sh compile.sh clean <file>
```

Primeri:

```
sh compile.sh rwx.c
sh compile.sh gcc -O2 -Wall rwx.c
sh compile.sh clean rwx
```

Ako korisnik ne zada kompajler, skripta koristi redosled:

1. CC iz okruženja,
2. CC iz config.txt,
3. podrazumevani cc

Ako korisnik ne zada CFLAGS, skripta koristi:

- za cl (MSVC): /W4 /O2
- za ostale kompajlere: -Wall -Wextra -O2

Skripta takođe podržava flagove -I, -L, -l, -D, -U i svake druge flagove koji počinju sa - ili /.

Nakon uspešnog kompajliranja, skripta ispisuje:

```
=====
Compiler: gcc
CFLAGS:    -Wall -Wextra -O2
LDFLAGS:
=====
-----
Success!
-----
-----
Run program with ./rwx
-----
```

Komanda clean uklanja binarni fajl.

3.12.3. Skripta install

Skripta install.sh (odnosno install.bat) kompajlira rwx.c i kopira binarni fajl u direktorijum BINDIR (podrazumevano PREFIX/bin na POSIX, PREFIX na Windows).

Tokom izvršavanja:

1. proverava da li rwx.c postoji,
2. poziva compile.sh sa QUIET=1 (kako ne bi ispisalo poruku Run program with ./rwx)
3. učitava config.txt,
4. Izvršava INSTALL_CMD,
5. ispisuje INSTALL_MSG i status uspeha.

Primer:

```
sh install.sh
```

Ako korisnik želi drugačiji PREFIX, može da ga zada kroz okruženje:

```
PREFIX=/opt/rwx sh install.sh
```

Na Windows, skripta automatski dodaje PREFIX u Path pomoću add_path_windows.bat.

3.12.4. Skripta uninstall

Skripta `uninstall.sh` (odnosno `uninstall.bat`) uklanja binarni fajl iz `BINDIR` (odnosno `PREFIX`) i ispisuje `UNINSTALL_MSG`.

Primer:

```
sh uninstall.sh
```

3.12.5. Skripta patch_code

Skripta `patch_code.sh` (odnosno `patch_code.bat`) je najvažnija skripta u suckless radnom toku. Ona omogućava korisniku da generiše i primeni zakrpe nad izvornim kodom bez ručnog pozivanja `diff` i `patch` alata.

Podržava dve komande:

```
sh patch_code.sh gen <originalni_fajl> <modifikovani_fajl> <izlazni.diff>  
sh patch_code.sh apply <patch_fajl> <ciljni_fajl>
```

Primeri:

```
sh patch_code.sh gen rxw.c rxw_cpp.c patch_cpp_support_1_0_0.diff  
sh patch_code.sh apply patch_cpp_support_1_0_0.diff rxw.c
```

Kako se generiše zakrpa unutar skripte:

1. proverava da li obe datoteke postoje,
2. bira između `diff` i `git diff` (koji god je dostupan),
3. ako fajlovi imaju Windows linije (CRLF), konvertuje ih u LF radi čiste razlike,
4. generiše unified diff.

Kako se primenjuje zakrpa unutar skripte:

1. kopira ciljni fajl u privremenu rezervu,
2. priprema patch fajl (uklanja specifično `git` zaglavlje, menja nazive fajlova),
3. pokušava da primeni zakrpu preko `patch` alata,
4. ako `patch` ne uspe, pokušava da primeni zakrpu preko `git apply`,
5. ako nijedan alat ne uspe, vraća rezervnu kopiju i ispisuje grešku.

Skripta takođe podržava `help`, `-h`, `--help`.

3.12.6. Skripta add_path_windows.bat

Ova skripta se koristi samo na Windows sistemima. Ona dodaje zadati direktorijum u sistemski Path ako taj direktorijum još nije prisutan. Koristi se unutar `install.bat` nakon kopiranja binarnog fajla, da bi korisnik mogao da pokreće `rxw` sa bilo koje lokacije.

3.12.7. Primer kompletnog rada sa skriptama

Naredni primer prikazuje tipičan workflow korisnika koji preuzima rwx projekat i želi da ga instalira:

Posix (Linux, MacOS, FreeBSD, OpenBSD, NetBSD):

```
# 1. Kloniranje repozitorijuma
git clone https://github.com/ZeljkoRasovic/rwx.git
cd rwx
# 2. Konfigurisanje (detekcija kompajlera)
sh configure.sh
# 3. Kompajliranje
sh compile.sh rwx.c
# 4. Pokretanje
./rwx
# 5. Instalacija
sudo sh install.sh
# 6. Uklanjanje
sudo sh uninstall.sh
```

Windows:

```
:: 1. Kloniranje repozitorijuma
git clone https://github.com/ZeljkoRasovic/rwx.git
cd rwx
:: 2. Konfigurisanje
configure.bat
:: 3. Kompajliranje
compile.bat rwx.c
:: 4. Pokretanje
rwx.exe
:: 5. Instalacija (traziti admin privilegije za dodavanje u Path)
install.bat
:: 6. Uklanjanje
uninstall.bat
```

Korisnik koji želi da proširi editor sopstvenom funkcionalnostima pravi modifikovanu kopiju rwx.c, zatim:

Posix (Linux, MacOS, FreeBSD, OpenBSD, NetBSD):

```
cp rwx.c rwx_modified.c
# edituje rwx_modified.c
# kada sve radi što hoćemo na rwx_modified.c
# radimo sledeće kako bi napravi i primeni zakrpu
sh patch_code.sh gen rwx.c rwx_modified.c moja_zakrpa.diff
sh patch_code.sh apply moja_zakrpa.diff rwx.c
sh compile.sh rwx.c
```

Windows:

```
cp rwx.c rwx_modified.c
:: edituje rwx_modified.c
:: kada sve radi što hoćemo na rwx_modified.c
:: radimo sledeće kako bi napravi i primeni zakrpu
.\patch_code.bat gen rwx.c rwx_modified.c moja_zakrpa.diff
.\patch_code.bat apply moja_zakrpa.diff rwx.c
.\compile.bat rwx.c
```

Ove skripte čine da suckless pristup bude pristupačan i korisnicima koji nisu napredni u korišćenju terminala.

3.13. Zakrpe za proširenje funkcionalnosti

U skladu sa suckless filozofijom, funkcionalnosti koje nisu neophodne za osnovni rad editora dodaju se kroz zakrpe (engl. patches). U ovom radu izrađene su dve referentne zakrpe, zakrpa za isticanje C++ sintakse, zakrpa za vim prečice.

3.13.1. Zakrpa za C++ sintaksu

Osnovna verzija editora rwx sadrži isticanje sintakse za C jezik.

Zakrpa patch_cpp_support_1_0_0.diff dodaje podršku za C++ ekstenzije i ključne reči.

Zakrpa dodaje sledeće strukture u rwx.c:

```
const char* const CPP_H_extensions[]=
{" .cpp", ".cc", ".cxx", ".hpp", ".hh", ".hxx", ".ii", ".ixx", ".ipp", ".cppm", ".tpp", NULL};
```

Niz CPP_H_keywords_len čuva dužine svake ključne reči, a CPP_H_keywords_syntax_color čuva boju za svaku ključnu reč (1, 2 ili 3).

Na kraju, zakrpa dodaje novi ulaz u bazu sintakse:

```
syntax H_DB[]=
{
{"C",C_H_extensions,C_H_keywords,C_H_keywords_len,C_H_keywords_syntax_color,"//","/*","*"/},
{"C++",CPP_H_extensions,CPP_H_keywords,CPP_H_keywords_len,CPP_H_keywords_syntax_color,"//","/*","*"/}
};
```

Time editor automatski prepoznaje .cpp, .cc, .cxx, .hpp, .hh, .hxx, .ii, .ixx, .ipp, .cppm, .tpp ekstenzije i ističe C++ ključne reči.

Primer primene:

```
sh patch_code.sh apply patch_cpp_support_1_0_0.diff rwx.c
sh compile.sh rwx.c
./rwx
```


3.13.2. Testni fajlovi za sintaksu

U repozitorijumu su priloženi dva testna fajla:

- `syntax_test.c`
- `syntax_test.cpp`

Oba fajla sadrže listu ključnih reči i operatora koje editor treba da prepozna. Korisnik može da otvori ove fajlove u `rwX` editoru i vizuelno proveriti da li su sve ključne reči pravilno obojene.

Primer testa:

```
# Kompajliranje editora sa C++ podrškom
sh compile.sh rwX.c
# Pokretanje editora na testnom fajlu
./rwX syntax_test.cpp
```

Ako je zakrpa za C++ sintaksu pravilno primenjena, sve C++ ključne reči u `syntax_test.cpp` bi trebalo da budu istaknute u odgovarajućim bojama.

Isto tako ako korisnik želi da testira da li je C sintaksa pravilno primenjena sve C ključne reči u `syntax_test.c` bi trebalo da budu istaknute u odgovarajućim bojama.

3.13.3. Zakrpa za vim prečice

Zakrpa za vim prečice dodaje editoru slično ponašanje kao editor vim. Ona uvodi dodatne modalne režime i prečice koje korisnici vim programa prepoznaju.

Zakrpa dodaje sledeće:

- Normal režim - kursor se kreće strelicama ili `h`, `j`, `k`, `l`, a tasteri za unos su deaktivirani dok korisnik ne pređe u Insert režim.
- Insert režim - aktivira se na `i`, `a`, `o`, `I`, `A`, `O`, a izlazi na `ESC`.
- Visual režim - aktivira se na `v` za selekciju karaktera, na `V` za selekciju linije.
- Komande - `dd` briše liniju, `p` lepi kopiran tekst, `:w` čuva fajl, `:q` izlazi, `:wq` čuva i izlazi.
- Navigacija - `w`, `b`, `e` za kretanje po rečima, `0` i `$` za početak i kraj linije, `gg` za početak fajla, `G` za kraj fajla.
- Zamena - `r` zamenjuje jedan karakter, `s` briše i menja karakter, `cw`, `ce`, `cb` menjaju reč.

Zakrpa koristi postojeću infrastrukturu editora (`actions`, `undo/redo`, `select_mode`) i ne uvodi nove zavisnosti. Nakon primene zakrpe i kompajliranja, editor idalje nema sve prečice kao pravi vim editor, mada editor sad ima na raspolaganju neke od prečica koje se nalaze u vim editoru, a da pri tome zadržava minimalističku bazu koda.

Primer primene:

```
sh patch_code.sh apply patch_vim_mode_1_0_0.diff rwX.c
sh compile.sh rwX.c
./rwX
```

4. Zaključak

U okviru ovog završnog rada uspešno su ostvareni svi projektni ciljevi definisani u zadatku. Razvijen je potpuno funkcionalan terminalski editor `rxw`, koji odlikuje jednostavnost izvornog koda i odsustvo suvišnih zavisnosti.

Pokazano je da se primenom `suckless` pristupa može dobiti izuzetno brz softver, gde krajnji korisnik zadržava potpunu kontrolu nad funkcionalnostima kroz proces primene zakrpa (engl. patches).

U programskom jeziku C implementirane su sledeće funkcionalnosti editora: rad u sirovom (engl. raw) režimu terminala, učitavanje i čuvanje fajlova, kretanje kursorom, uređivanje teksta, prikaz statusne linije, pretraga i zamena, `undo/redo`, file browser, isticanje sintakse određenih programskih jezika, modalno uređivanje, podrška za miš, bracketed paste, detekcija spoljašnje izmene fajla i kopiranje putem OSC52 escape sekvence.

Uz editor dolaze i pomoćne skripte (`configure`, `compile`, `install`, `uninstall`, `patch_code`) koje olakšavaju korisniku konfigurisanje, kompajliranje, instalaciju, uklanjanje i proširenje funkcionalnosti editora. Izrađene su dve referentne zakrpe, zakrpa za isticanje C++ sintakse i zakrpa za vim prečice. U dokumentaciji je dat i primer kako korisnik samostalno može da generiše i primeni sopstvenu zakrpu.

Merenja performansi pokazuju linearno rastući trošak memorije sa brojem linija. U poređenju sa drugim editorima poput `neovim`, `rxw` je znatno manji i efikasniji sa memorijom.

Mada ovaj način projektovanja softvera takođe ima i svoje nedostatke.

Nedostaci ovog načina projektovanje softvera su:

- obaveza rekompajliranja,
- konflikti pri primeni više zakrpa na isti kod,
- održavanje zakrpa pri promeni koda editora,
- nedostatak izolacije (greška u zakrpi ruši ceo editor),
- viši prag znanja za korisnika.

Budući pravci razvoja projekta obuhvataju dodavanje podrške za dodatne programske jezike kroz nove zakrpe (engl. patches), dodavanje podrške za više bafera fajlova (split prikaz) i macro sistem.

Literatura

- [1] P. J. Salter. "Build Your Own Text Editor." Internet: <https://viewsourcecode.org/snaptoken/kilo/> [Aug. 27, 2026].
- [2] suckless.org. "Software that sucks less, Philosophy." Internet: <https://suckless.org/philosophy/> [Aug. 27, 2026].
- [3] A. S. Tanenbaum and H. Bos. Modern Operating Systems, 4th ed. Upper Saddle River, NJ: Pearson, 2014.
- [4] Ž. Rašović. "rwx: Minimal terminal code editor." Internet: <https://github.com/ZeljkoRasovic/rwx> [Aug. 27, 2026].
- [5] B. W. Kernighan and D. M. Ritchie. The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [6] B. Moolenaar. "Vim - Vi IMproved." Internet: <https://www.vim.org/> [Aug. 27, 2026].
- [7] Neovim Core Team. "Neovim: Hyperextensible Vim-based text editor." Internet: <https://neovim.io/> [Aug. 27, 2026].
- [8] GNU Project. "GNU nano" Internet: <https://www.nano-editor.org/> [Aug. 27, 2026].
- [9] IEEE. "IEEE Standard for Information Technology - Portable Operating System Interface (POSIX) Base Specifications." IEEE Std 1003.1-2017, 2018.

5. Prilozi

5.1. Štampani materijali

Zbog obima rada, potpuni izvorni kod nije odštampan.

5.2. Elektronski materijali

Elektronski prilog sadrži završni rad u DOCX i PDF formatu, PowerPoint prezentaciju za odbranu, konačnu verziju rwx projekta sa svim skriptama i referentnim zakrpama (engl. patches).